Aprendizaje por Refuerzo

David Valera López
d.valeralopez1@um.es

Lucía Sánchez Montes Gómez
lucia.s1@um.es

15 de marzo de 2026

Resumen

Este trabajo estudia el comportamiento de distintos algoritmos de aprendizaje por refuerzo en problemas de toma de decisiones bajo incertidumbre. En la primera parte se analiza el problema del bandido de k-brazos, evaluando diferentes estrategias de selección de acciones que buscan equilibrar exploración y explotación, entre ellas ϵ-Greedy, ϵ-Decay, Softmax y los métodos basados en límites de confianza superior (UCB1 y UCB2). Los algoritmos se comparan experimentalmente en tres entornos estocásticos con características distintas de varianza y escala de recompensas: distribuciones de Bernoulli, Binomial y Normal. Para evaluar su rendimiento se utilizan métricas como la recompensa promedio, el porcentaje de selección del brazo óptimo, el *regret* acumulado y la distribución de selecciones por brazo.

En la segunda parte del trabajo se aborda el aprendizaje por refuerzo en entornos más complejos modelados como procesos de decisión de Markov (MDP). Se comparan métodos tabulares y técnicas de aproximación de funciones en dos entornos de la librería *Gymnasium*: *Taxi-v3*, con un espacio de estados discreto, y *Acrobot-v1*, con observaciones continuas que requieren aproximación de funciones. En estos escenarios se analizan algoritmos como Monte Carlo *On-Policy* y *Off-Policy*, SARSA, Q-Learning, SARSA semigradiente con *tile coding* y Deep Q-Learning (DQN).

Los resultados experimentales muestran que los métodos basados en límites de confianza superior, especialmente UCB1, presentan el comportamiento más robusto frente a cambios en la distribución y la escala de las recompensas en el problema del bandido, logrando minimizar el *regret* acumulado y converger de forma consistente hacia la acción óptima. En los entornos complejos, los métodos tabulares demuestran un buen rendimiento en espacios discretos como *Taxi-v3*, donde algoritmos basados en diferencia temporal como SARSA y Q-Learning consiguen aprender políticas eficientes. En el entorno continuo *Acrobot-v1*, los métodos basados en aproximación de funciones, como el SARSA semigradiente y Deep Q-Learning, muestran una mayor capacidad de aprendizaje al poder generalizar entre estados similares. Finalmente, el análisis experimental de la cota teórica de Lai-Robbins permite explicar por qué algunas estrategias pueden aparentar superar este límite en horizontes temporales finitos. En conjunto, el estudio evidencia la importancia de gestionar adecuadamente la incertidumbre y de seleccionar algoritmos adecuados al tipo de entorno para diseñar agentes de aprendizaje por refuerzo eficientes.

Índice general

1. Problema del Bandido de k-brazos **3**

1.1. Introducción . . . 3

1.2. Desarrollo . . . 3

1.2.1. Definición formal y contexto del problema . . . 3

1.2.2. Enfoques existentes y métodos implementados . . . 3

1.2.3. Justificación y aportación empírica del estudio . . . 4

1.3. Algoritmos . . . 4

1.3.1. Estrategias heurísticas: ϵ-Greedy y ϵ-Decay . . . 4

1.3.2. Estrategias probabilísticas: Softmax . . . 4

1.3.3. Estrategias optimistas: UCB1 y UCB2 . . . 5

1.4. Evaluación y Experimentos . . . 5

1.4.1. Configuración Experimental . . . 5

1.4.2. Resultados: Entorno de Bernoulli . . . 6

1.4.3. Resultados: Entorno Binomial . . . 7

1.4.4. Resultados: Entorno Normal . . . 9

1.4.5. Análisis Experimental de la Cota Teórica de Lai-Robbins . . . 10

1.5. Conclusiones . . . 12

2. Aprendizaje en Entornos Complejos **14**

2.1. Introducción . . . 14

2.2. Desarrollo . . . 14

2.2.1. Formulación del problema . . . 14

2.2.2. Métodos tabulares . . . 14

2.2.3. Métodos aproximados . . . 15

2.2.4. Entornos utilizados . . . 15

2.3. Algoritmos . . . 15

2.3.1. Monte Carlo On-Policy . . . 15

2.3.2. Monte Carlo Off-Policy 16
2.3.3. SARSA 16
2.3.4. Q-Learning 16
2.3.5. SARSA semigradiente con *tile coding* 16
2.3.6. Deep Q-Learning 17
2.4. Evaluación/Experimentos 18
2.4.1. Métricas de Desempeño 18
2.4.2. Hiperparámetros empleados 18
2.4.3. Resultados en `Taxi-v3` 19
2.4.4. Resultados en `Acrobot-v1` 20
2.5. Conclusiones 21

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

El problema del bandido multibrazo (*k-armed bandit*) constituye uno de los paradigmas fundamentales dentro del campo del Aprendizaje por Refuerzo. Este problema modela una situación en la que un agente debe elegir repetidamente entre K opciones o acciones diferentes, cada una de las cuales proporciona una recompensa numérica extraída de una distribución de probabilidad desconocida. Su relevancia radica en que ilustra de manera pura el clásico dilema entre la *exploración* (probar distintas acciones para descubrir su valor real) y la *explotación* (seleccionar la acción que actualmente se percibe como la mejor para maximizar el beneficio a corto plazo).

La motivación principal para el estudio de este problema es su aplicabilidad directa en escenarios del mundo real donde se requiere la toma de decisiones bajo incertidumbre. Problemas como la asignación de tratamientos en ensayos clínicos, la optimización de campañas publicitarias mediante testeo A/B, o los sistemas de recomendación modernos, pueden modelarse utilizando esta arquitectura. Comprender cómo las diferentes estrategias matemáticas abordan el equilibrio entre explorar y explotar es importante, ya que una estrategia ineficiente puede derivar en la pérdida de recursos valiosos o en un coste de oportunidad significativo.

En este contexto, el objetivo de este informe es presentar un estudio empírico exhaustivo y comparativo del rendimiento de diversas familias de algoritmos de selección de acciones, tales como los enfoques heurísticos (ϵ-Greedy y sus variantes con decaimiento), de selección probabilística (Softmax) y los métodos optimistas basados en límites de confianza superior (UCB1 y UCB2). A través de la experimentación práctica, se pretende evaluar y contrastar el comportamiento, la convergencia y el *regret* (arrepentimiento) acumulado de estas técnicas frente a tres entornos estacionarios con naturalezas probabilísticas diferentes: distribuciones de Bernoulli, Binomiales y Normales.

El resto de este documento está organizado de la siguiente manera. En primer lugar, se describirá el marco teórico y la configuración del entorno de experimentación, detallando los algoritmos implementados y las métricas de evaluación utilizadas. Posteriormente, se expondrán los resultados obtenidos empíricamente para cada uno de los entornos probabilísticos (Bernoulli, Binomial y Normal), acompañados de sus respectivas gráficas y análisis analíticos. Finalmente, el informe concluirá con una síntesis de los descubrimientos más relevantes, determinando qué estrategias resultan más robustas y eficientes en función de las características de las recompensas.

1.2. Desarrollo

1.2.1. Definición formal y contexto del problema

El problema del bandido multibrazo (*K-armed bandit*) es una abstracción clásica en el Aprendizaje por Refuerzo que modela la toma de decisiones secuenciales bajo incertidumbre. Formalmente, se define un agente que, en cada instante de tiempo $t = 1, 2, \ldots, T$, debe seleccionar una acción A_t de entre un conjunto de K acciones posibles (o "brazos"). Al ejecutar la acción elegida, el entorno devuelve una recompensa numérica R_t, extraída de una distribución de probabilidad subyacente y estacionaria que el agente desconoce.

El objetivo fundamental del agente es maximizar la suma total de las recompensas obtenidas a lo largo del tiempo. Para ello, debe estimar el valor esperado real de cada acción, denotado como $q_*(a) = \mathbb{E}[R_t|A_t = a]$. Dado que el agente solo conoce las recompensas de las acciones que ha probado, se enfrenta al clásico dilema entre la *exploración* (probar acciones menos frecuentes para mejorar la estimación de su valor) y la *explotación* (seleccionar la acción con el mayor valor estimado actual para asegurar una recompensa inmediata alta).

1.2.2. Enfoques existentes y métodos implementados

A lo largo de la literatura, se han propuesto múltiples enfoques para resolver este balance. Las aproximaciones más elementales incluyen métodos de selección pura, como el enfoque *Greedy*, o estrategias heurísticas como ϵ-Greedy. Sin embargo, explorar de forma completamente aleatoria presenta ineficiencias, lo que motivó la aparición de selecciones probabilísticas ponderadas como Softmax, y posteriormente, enfoques basados en el "optimismo frente a la incertidumbre" respaldados por la cota teórica de Lai y Robbins, dando lugar a los algoritmos UCB (*Upper Confidence Bound*).

En este trabajo empírico, se ha implementado y analizado los siguientes métodos:

- **ϵ-Greedy y sus variantes:** Seleccionan la mejor acción conocida con probabilidad $1 - \epsilon$, y exploran de forma uniforme al azar con probabilidad ϵ. Su variante ϵ-Decay reduce progresivamente esta tasa para converger hacia la explotación.

- **Softmax:** A diferencia de la exploración uniforme de ϵ-Greedy, Softmax asigna una probabilidad de selección a cada acción basada en la distribución de Gibbs. Utiliza un parámetro de temperatura (τ) que, al ser alto, promueve una exploración casi equiprobable y, al disminuir, concentra exponencialmente la probabilidad en las acciones con mayor estimación de valor, evitando explorar los brazos que ya se saben deficientes.
- **UCB1:** Selecciona la acción que maximiza $Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}}$, donde $Q_t(a)$ es el valor estimado, $N_t(a)$ es el número de veces que se ha seleccionado el brazo, y c es un parámetro que regula el límite de confianza superior.
- **UCB2:** Una variante más sofisticada que divide la exploración en épocas temporales, reduciendo la frecuencia de los cálculos matemáticos e intentando optimizar la precisión asintótica a largo plazo.

1.2.3. Justificación y aportación empírica del estudio

Si bien el comportamiento teórico de estos algoritmos está ampliamente documentado en la literatura, la mayoría de los enfoques introductorios asumen entornos con recompensas acotadas entre 0 y 1 (como las distribuciones de Bernoulli). El presente trabajo se diferencia y complementa la teoría clásica al plantear un estudio que somete a la misma batería de algoritmos a tres entornos de distinta naturaleza estocástica: Bernoulli, Binomial y Normal.

Esta aproximación práctica permite no solo corroborar la superioridad teórica de los métodos basados en límites de confianza frente a los heurísticos o probabilísticos puros, sino también evidenciar vulnerabilidades no triviales. Por ejemplo, en los entornos binomiales y normales, la magnitud y varianza de las recompensas alteran drásticamente el peso del término de incertidumbre en UCB o la influencia de la temperatura en Softmax. Nuestro trabajo ilustra empíricamente la necesidad crítica de escalar y ajustar los hiperparámetros de exploración (c, α o τ) en proporción a la escala de las recompensas, demostrando que no existe un algoritmo universalmente superior sin una adaptación correcta al contexto del problema.

1.3. Algoritmos

La selección de los algoritmos para este estudio no es arbitraria, obedece a la necesidad de evaluar el espectro completo de estrategias que abordan el dilema de exploración frente a explotación en el problema del bandido multibrazo. Comenzamos con enfoques heurísticos básicos (ϵ-Greedy), evolucionamos hacia métodos basados en distribuciones de probabilidad (Softmax) y concluimos con estrategias deterministas basadas en límites de confianza superior (UCB1 y UCB2). Esta progresión nos permite analizar cómo el incremento en la sofisticación matemática impacta en la adaptabilidad del agente frente a entornos estocásticos diversos (Bernoulli, Binomial y Normal).

A continuación, se describen en detalle los pasos de cada método y se presenta su pseudocódigo asociado. Todos los algoritmos comparten un mecanismo común de actualización del valor esperado $Q(A_t)$ mediante una regla de actualización incremental de la media.

1.3.1. Estrategias heurísticas: ϵ-Greedy y ϵ-Decay

El algoritmo ϵ-Greedy es una de las estrategias más simples y efectivas. En cada instante de tiempo t, el agente selecciona la acción que tiene el valor estimado más alto (explotación) con una probabilidad de $1-\epsilon$. Sin embargo, con una probabilidad ϵ, el agente elige una acción al azar de forma uniforme entre todas las disponibles (exploración).

Para evitar explorar de forma indefinida, se suele implementar una variante denominada ϵ-Decay, donde el valor de ϵ disminuye progresivamente con el tiempo. Esto permite al algoritmo explorar intensamente en las primeras iteraciones y, conforme adquiere más conocimiento, converger hacia una explotación casi pura. El comportamiento base se describe en el **Algorithm 1**.

Algorithm 1 ϵ-Greedy Bandit Algorithm

Require: $\epsilon \in [0,1]$, K arms, total steps T
Initialize $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$ for all $a \in \{1,\dots,K\}$
for $t = 1$ to T **do**
  Draw a random number $p \sim U(0,1)$
  if $p < \epsilon$ **then**
    Select random action $A_t \in \{1,\dots,K\}$
  else
    Select action $A_t \leftarrow \arg\max_a Q(a)$
  end if
  Obtain reward R_t from environment
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

1.3.2. Estrategias probabilísticas: Softmax

A diferencia de ϵ-Greedy, que explora todas las acciones no óptimas con la misma probabilidad, Softmax utiliza una distribución de Gibbs para ponderar la probabilidad de elegir cada brazo en función de su valor estimado actual. Utiliza un hiperparámetro de temperatura (τ). Si τ es alto, las probabilidades tienden a ser uniformes (mucha exploración), si τ es cercano a cero, la probabilidad se concentra casi exclusivamente en el brazo con mayor $Q(a)$, comportándose de forma similar a un algoritmo puramente avaricioso. El detalle se muestra en el **Algorithm 2**.

Algorithm 2 Softmax Bandit Algorithm

Require: Temperature $\tau > 0$, K arms, total steps T
Initialize $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$ for all $a \in \{1,\dots,K\}$
for $t = 1$ to T **do**
  for $a = 1$ to K **do**
    $\pi(a) \leftarrow \frac{\exp(Q(a)/\tau)}{\sum_{i=1}^{K}\exp(Q(i)/\tau)}$
  end for
  Select action A_t according to probabilities $\pi(a)$
  Obtain reward R_t from environment
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

1.3.3. Estrategias optimistas: UCB1 y UCB2

Los algoritmos *Upper Confidence Bound* (UCB) abordan el problema calculando un límite superior de confianza para la estimación del valor de cada acción. UCB1 (**Algorithm 3**) selecciona siempre la acción que maximiza la suma de su valor estimado $Q(a)$ y un término de incertidumbre $\sqrt{\ln t/N(a)}$. Este término crece con el tiempo global t pero disminuye cada vez que se selecciona el brazo en cuestión ($N(a)$), forzando así la exploración de brazos poco probados. El parámetro c controla el grado de exploración y es crucial ajustarlo en función de la escala de las recompensas del entorno.

Algorithm 3 UCB1 Algorithm

Require: Parameter $c > 0$, K arms, total steps T
Initialize $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$ for all $a \in \{1, \ldots, K\}$
for $t = 1$ to K **do** ▷ Initialization: pull each arm once
  Select action $A_t \leftarrow t$
  Obtain reward R_t
  $N(A_t) \leftarrow 1$, $Q(A_t) \leftarrow R_t$
end for
for $t = K + 1$ to T **do**
  Select action $A_t \leftarrow \arg\max_a \left(Q(a) + c\sqrt{\frac{\ln t}{N(a)}}\right)$
  Obtain reward R_t from environment
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

Algorithm 4 UCB2 Algorithm

Require: Parameter $\alpha > 0$, K arms, total steps T
Initialize $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$, $r(a) \leftarrow 0$ for all $a \in \{1, \ldots, K\}$
Define function $\tau(x) = \lceil (1 + \alpha)^x \rceil$
for $t = 1$ to K **do** ▷ Initialization: pull each arm once
  Select action $A_t \leftarrow t$
  Obtain reward R_t
  $N(A_t) \leftarrow 1$, $Q(A_t) \leftarrow R_t$, $r(A_t) \leftarrow 0$
end for
$t \leftarrow K$
while $t < T$ **do**
  Select action $A \leftarrow \arg\max_a \left(Q(a) + \sqrt{\frac{(1+\alpha)\ln(e \cdot t/\tau(r(a)))}{2\tau(r(a))}}\right)$
  $\Delta \leftarrow \tau(r(A) + 1) - \tau(r(A))$ ▷ Number of pulls in this epoch
  for $i = 1$ to Δ **do**
    if $t \geq T$ **then break**
    end if
    Obtain reward R from environment
    $N(A) \leftarrow N(A) + 1$
    $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}(R - Q(A))$ ▷ Incremental mean update
    $t \leftarrow t + 1$
  end for
  $r(A) \leftarrow r(A) + 1$ ▷ Advance epoch for the chosen arm
end while

Por su parte, la variante **UCB2** (**Algorithm 4**) introduce el concepto de *épocas* para reducir la frecuencia con la que se reevalúan las matemáticas de incertidumbre. En lugar de actualizar y elegir una nueva acción en cada turno t, cuando UCB2 selecciona un brazo, lo juega repetidamente durante toda una época.

Para lograr esto, el algoritmo mantiene un contador de épocas $r(a)$ para cada brazo. La duración de estas épocas crece de forma exponencial según la función $\tau(r) = \lceil (1 + \alpha)^r \rceil$, donde $\alpha > 0$ es un hiperparámetro configurable. El número de veces que se tira un brazo en su época actual viene dado por la diferencia $\tau(r+1) - \tau(r)$. Aunque su implementación requiere mantener más variables de estado que UCB1, esta estrategia de bloques temporales busca mejorar la cota teórica del *regret* a largo plazo, ahorrando coste computacional, a cambio de una menor agilidad de adaptación frente a fluctuaciones en las etapas iniciales del entorno.

1.4. Evaluación y Experimentos

En esta sección se detallan las pruebas empíricas realizadas para evaluar el desempeño de los algoritmos implementados. A diferencia de otros enfoques presentes en la literatura que segmentan el análisis por familia algorítmica, este estudio evalúa el comportamiento transversal de todos los métodos frente a distintas naturalezas probabilísticas del entorno. El objetivo es analizar la convergencia, la estabilidad y la sensibilidad a la escala de las recompensas.

1.4.1. Configuración Experimental

Entornos de Prueba

Para evaluar los algoritmos se diseñaron tres entornos de simulación del problema del bandido multibrazo (K brazos), cada uno caracterizado por una distribución de recompensa distinta para analizar el impacto de la varianza y la magnitud:

- **Distribución de Bernoulli:** Simula recompensas binarias o tasas de éxito/fracaso, acotando estrictamente los valores en el intervalo $\{0, 1\}$.
- **Distribución Binomial:** Introduce una varianza discreta y amplía el rango numérico de los premios posibles, simulando la suma de múltiples ensayos de Bernoulli independientes.
- **Distribución Normal (Gaussiana):** Representa el entorno más complejo, con recompensas continuas extraídas de $\mathcal{N}(\mu, \sigma^2)$, introduciendo una alta variabilidad empírica que incluye recompensas tanto positivas como negativas.

Herramientas y Reproducibilidad

Los experimentos se han implementado en Python, ejecutándose a través de *Jupyter Notebooks*. Para asegurar la total reproducibilidad de los experimentos y garantizar una comparativa justa y directa, se ha fijado una semilla aleatoria (*seed*) constante para todos los generadores pseudoaleatorios (`numpy.random`). De esta forma, todos los algoritmos se enfrentan exactamente a la misma secuencia de recompensas latentes en cada ejecución.

Métricas de Desempeño

Para cuantificar la calidad de las estrategias, se han monitorizado cuatro métricas fundamentales a lo largo de un horizonte temporal T fijado, promediando los resultados de múltiples ejecuciones independientes:

1. **Porcentaje de Selección Óptima:** Proporción de veces que el agente selecciona el brazo con la mayor esperanza matemática real (μ^*).
2. **Recompensa Promedio:** Evolución del valor medio obtenido en cada paso de tiempo, indicando la rentabilidad directa de la política.
3. **Rechazo Acumulado (Regret):** Diferencia acumulada entre la recompensa teórica máxima (política óptima) y la obtenida empíricamente por el agente. El objetivo matemático es minimizar esta curva.
4. **Estadísticas por Brazo:** Histograma que contabiliza el número total de veces que el agente ha seleccionado cada uno de los K brazos durante el experimento. Esta métrica permite visualizar de forma directa si la política convergió exitosamente al brazo óptimo o si quedó atrapada explotando óptimos locales.

1.4.2. Resultados: Entorno de Bernoulli

En la distribución de Bernoulli, las recompensas son estrictamente binarias $\{0, 1\}$, lo que simula un entorno de éxito o fracaso. Al estar las recompensas acotadas en este intervalo, se mitigan los problemas de escala, permitiendo observar el comportamiento teórico de los algoritmos y la eficiencia de sus respectivas reglas de actualización.

Recompensa Promedio

En la Figura 1.1 se muestra la evolución de la recompensa promedio. Los métodos basados en límites de confianza superior (UCB) demuestran una superioridad inmediata, convergiendo hacia el máximo global de forma estable. Resulta notable el comportamiento de ϵ-Decay: aunque incurre en un déficit de recompensa inicial debido a su fase de exploración intensiva, su decaimiento progresivo le permite describir una curva de aprendizaje asintótica que termina igualando a los métodos UCB.

Por el contrario, el algoritmo ϵ-Greedy evidencia su principal limitación estructural: una tasa de exploración fija (10 %) que actúa como una cota superior, estabilizando su recompensa permanentemente por debajo de los algoritmos óptimos. Finalmente, Softmax presenta el rendimiento más deficiente en este escenario; su formulación probabilística le impide descartar de forma definitiva las opciones subóptimas, lo que se traduce en un comportamiento más estocástico y en la recompensa media más baja del experimento.

Porcentaje de Selección del Brazo Óptimo

La capacidad para identificar y explotar la mejor decisión se evalúa en la Figura 1.2. En la parte inferior, Softmax y ϵ-Greedy se ven limitados por sus topes estructurales (la dispersión probabilística y la exploración constante, respectivamente), viéndose incapacitados para aislar el máximo global de manera absoluta. En el tramo intermedio, ϵ-Decay logra

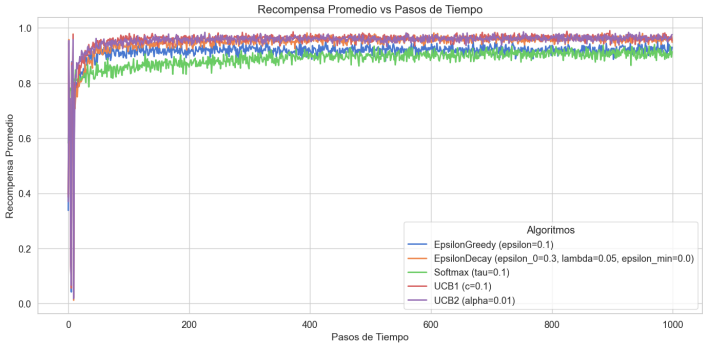

Figura 1.1: Evolución de la recompensa promedio en el entorno Bernoulli.

estabilizar el aprendizaje, convergiendo hacia una tasa de selección óptima cercana al 70 %.

El nivel superior está dominado en exclusiva por la familia UCB. Mientras UCB1 es el más eficiente en el corto y medio plazo gracias a su rápida adaptación, UCB2 demuestra las ventajas de su diseño por épocas: aunque sacrifica la velocidad de convergencia inicial, logra una mayor precisión asintótica en la segunda mitad del experimento, siendo el único método capaz de superar el 80 % de selecciones óptimas.

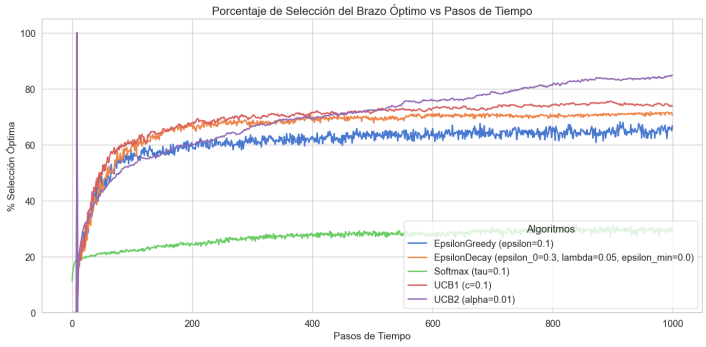

Figura 1.2: Evolución del porcentaje de selección del brazo óptimo en el entorno Bernoulli.

Evolución del Regret Acumulado

El *regret* (arrepentimiento) acumulado, ilustrado en la Figura 1.3, termina de demostrar la ineficiencia de las heurísticas clásicas a largo plazo. Softmax y ϵ-Greedy presentan un crecimiento estrictamente lineal, penalizados por una exploración subóptima constante.

En un punto intermedio se sitúa ϵ-Decay. Si bien este método logra aplanar la curva conforme avanza el tiempo, el alto volumen de selecciones subóptimas realizadas durante su fase exploratoria inicial genera un arrepentimiento acumulado que resulta irrecuperable respecto a los algoritmos líderes.

El dominio empírico corresponde a la familia UCB, cuyas curvas de *regret* exhiben un crecimiento logarítmico, manteniéndose casi horizontales. Aunque UCB2 mostró mayor precisión asintótica al final del experimento, **UCB1** minimiza de forma más efectiva el *regret* global, ya que su rápida adaptación inicial le permite evitar selecciones erróneas en los primeros

compases del problema.

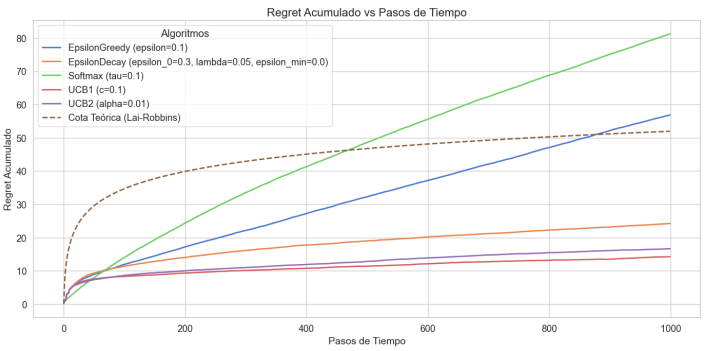

Figura 1.3: Evolución del regret acumulado en el entorno Bernoulli..

Estadísticas por Brazo (Distribución de Selecciones)

La Figura 1.4 revela la eficiencia interna de las políticas. En el grupo de menor rendimiento, el sesgo de Softmax es evidente: el algoritmo experimenta una severa convergencia subóptima, llegando a seleccionar el Brazo 1 en 149.000 ocasiones, superando las 135.000 selecciones del Brazo 8 (el óptimo real). Por su parte, ϵ-Greedy evidencia el coste de su exploración obligatoria, registrando más de 5.000 selecciones distribuidas de forma uniforme en cada una de las peores opciones del entorno.

En la zona media, ϵ-Decay demuestra la eficacia de su tasa de exploración decreciente: logra reducir drásticamente el ruido de los brazos subóptimos y concentra más de 331.000 selecciones en el brazo óptimo, superando ampliamente a su variante estática.

Los métodos UCB confirman su superioridad al discriminar la incertidumbre eficazmente, limitando las visitas a los peores brazos a cifras marginales (aproximadamente 500 selecciones en los brazos 5 y 9). En esta métrica, **UCB2** alcanza el máximo nivel de explotación del brazo óptimo de todo el estudio, con cerca de 348.000 selecciones.

Conclusiones del Entorno de Bernoulli

En definitiva, para entornos estacionarios de naturaleza Bernoulli, los enfoques heurísticos basados en límites de confianza representan la mejor opción. Mientras que algoritmos como ϵ-Greedy constituyen una base funcional, el mantenimiento de una tasa de exploración fija ejerce de barrera, impidiendo la convergencia absoluta. De forma análoga, Softmax ha demostrado ser el modelo menos robusto en este escenario debido a su incapacidad para descartar de forma definitiva las opciones subóptimas sin una calibración dinámica de la incertidumbre.

La introducción de dinamismo temporal a través de ϵ-Decay aporta una mejora sustancial en la estabilización de la política. Sin embargo, el coste de exploración asumido en las etapas iniciales penaliza su métrica global de *regret*. Por consiguiente, la implementación de **UCB1** o **UCB2** constituye la aproximación matemática más eficiente para maximizar la recompensa acumulada y garantizar la convergencia hacia la decisión óptima.

1.4.3. Resultados: Entorno Binomial

El entorno Binomial supone un incremento significativo en la complejidad del problema. Al modelar la recompensa como la suma de múltiples ensayos de Bernoulli independientes, no solo se introduce una mayor varianza empírica, sino que la escala de las recompensas se expande drásticamente (alcanzando valores máximos cercanos a 20 en nuestra configuración).

Recompensa Promedio

La Figura 1.6 ilustra la evolución de la recompensa media. El cambio de escala y varianza penaliza severamente al algoritmo Softmax. A pesar de utilizar su temperatura óptima calibrada para este entorno ($\tau = 5{,}0$), el método sufre un colapso en su rendimiento, estabilizándose prematuramente en un nivel muy deficiente (en torno a 16.5). Esto demuestra que la distribución de Gibbs encuentra dificultades para separar probabilísticamente valores esperados cuando la escala de las recompensas crece, requiriendo temperaturas extremas que terminan rompiendo el equilibrio exploración-explotación.

Por el contrario, UCB1 y UCB2 exhibe una robustez muy buena, alcanzando el techo de la recompensa casi de forma instantánea. ϵ-Decay describe nuevamente una curva de aprendizaje asintótica muy pronunciada: su exploración inicial lo penaliza gravemente en los primeros 200 pasos, pero su decaimiento programado le permite casi igualar a la familia UCB en las etapas tardías. La variante ϵ-Greedy óptima se mantiene estable, pero topada por su porcentaje fijo de exploración, aunque con unos resultados casi tan buenos como los de la familia UCB.

Porcentaje de Selección del Brazo Óptimo

La métrica de selección óptima (Figura 1.7) revela que UCB1 domina de manera absoluta, rozando el 100 % de precisión muy tempranamente. Sin embargo, **UCB2** sufre una degradación notable respecto a su excelente desempeño en Bernoulli, estancándose en una meseta cercana al 80 %. La división por épocas de UCB2 parece forzar una exploración excesiva de brazos subóptimos cuando la varianza del entorno es mayor, lo que retrasa su convergencia en comparación con la agilidad de actualización paso a paso de UCB1.

ϵ-Decay protagoniza la remontada más destacable, cruzando la línea de ϵ-Greedy alrededor del paso 400 y estabilizándose por encima del 90 %. Softmax, víctima de su sensibilidad a la magnitud de los premios, apenas logra identificar el brazo óptimo en un 25 % de las ocasiones, un comportamiento casi equiparable a una selección aleatoria, que es a lo que se acerca con un valor tan alto de temperatura, pero que curiosamente, ha sido el que mejor ha funcionado dentro su familia.

Evolución del Regret Acumulado

La curva de *regret* (Figura 1.8) cuantifica el coste real de las políticas. Softmax presenta una divergencia lineal, acumulando el mayor arrepentimiento del estudio (superando la cota de 2500).

El caso de ϵ-Decay es curioso, aunque logra una excelente selección óptima al final del experimento, su fase exploratoria

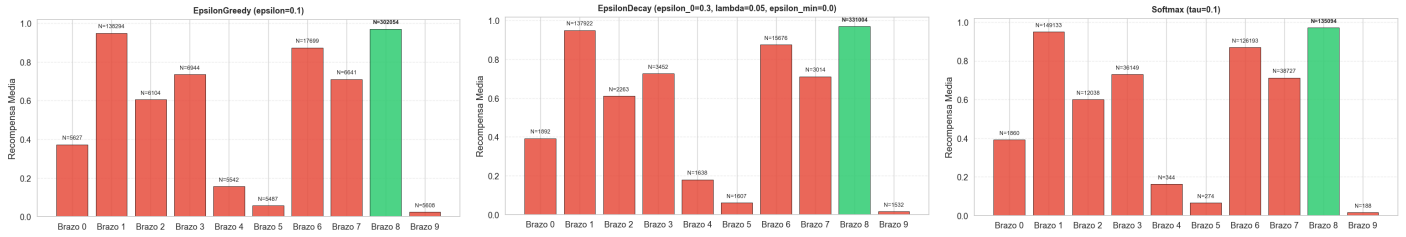

Figura 1.4: Histograma de estadísticas por brazo en el entorno Bernoulli. Se contrasta el récord de precisión de UCB2 frente a la convergencia a óptimos locales en Softmax.

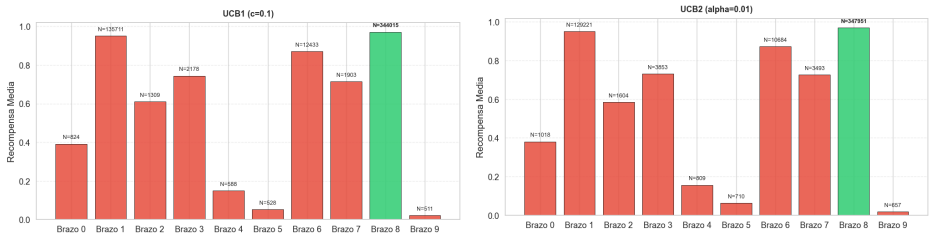

Figura 1.5: Histograma de estadísticas por brazo en el entorno Bernoulli (continuación).

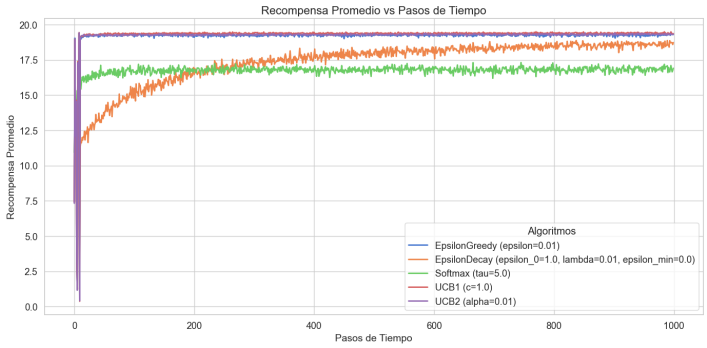

Figura 1.6: Evolución de la recompensa promedio en el entorno Binomial.

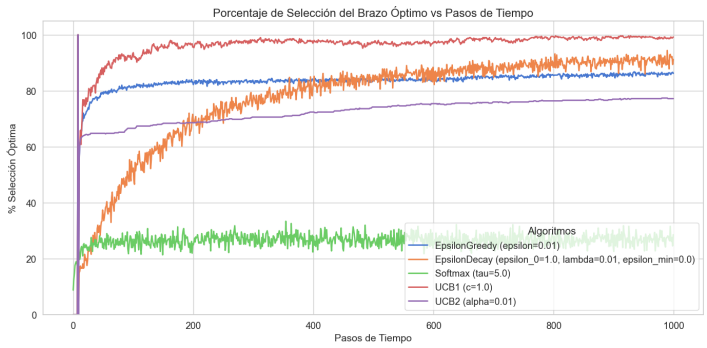

Figura 1.7: Evolución del porcentaje de selección del brazo óptimo en el entorno Binomial.

inicial genera un *regret* muy grande irrecuperable en el corto plazo (superior a 2000). Las estrategias UCB se mantienen como las más eficientes. UCB1 supera con creces a UCB2, minimizando de forma magistral el *regret* global y consolidándose como el método más adaptativo frente al aumento de la varianza, siendo este el que más se acerca a la cota inferior teórica.

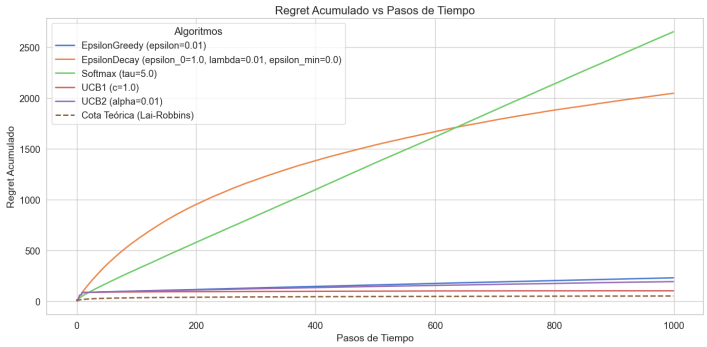

Figura 1.8: Evolución del regret acumulado en el entorno Binomial.

Estadísticas por Brazo (Distribución de Selecciones)

El histograma de selecciones (Figura 1.9) confirma visualmente las conclusiones anteriores. El brazo óptimo real es el Brazo 8. UCB1 demuestra un nivel de explotación óptima innegable, acumulando más de 478.000 selecciones en dicho brazo y minimizando el ruido en el resto, en algunos incluso solo tiene 500 selecciones, que es el mínimo obligatorio debido a la inicialización incial de los brazos.

En el otro extremo, Softmax distribuye su atención de forma errática. A pesar de tratarse de su mejor parametrización, los valores Q de los brazos 1, 6 y 7 resultan probabilísticamente demasiado cercanos al brazo 8 bajo la función de Gibbs, lo que lleva al agente a atascarse en decisiones subóptimas de alto valor durante cientos de miles de iteraciones combinadas.

Conclusiones del Entorno Binomial

La principal conclusión de este escenario es la extrema vulnerabilidad de los métodos probabilísticos (como Softmax) ante cambios drásticos en la magnitud de las recompensas. Incluso empleando el hiperparámetro óptimo hallado durante la fase de validación, la función exponencial es incapaz de discrimi-

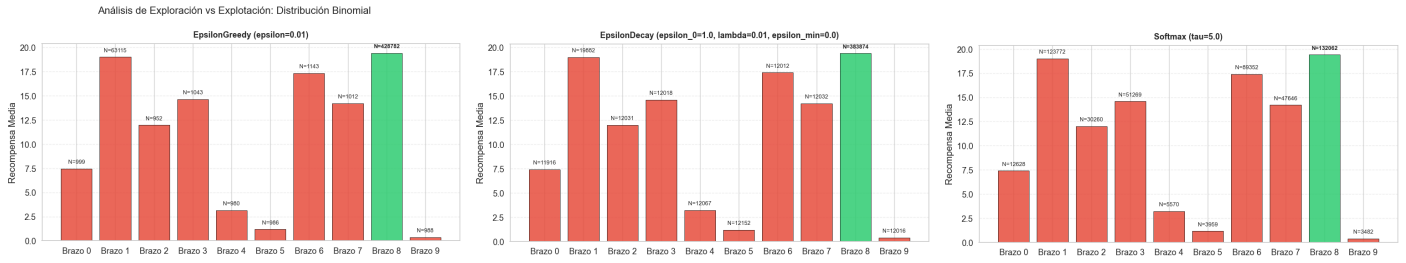

Figura 1.9: Histograma de estadísticas por brazo en el entorno Binomial.

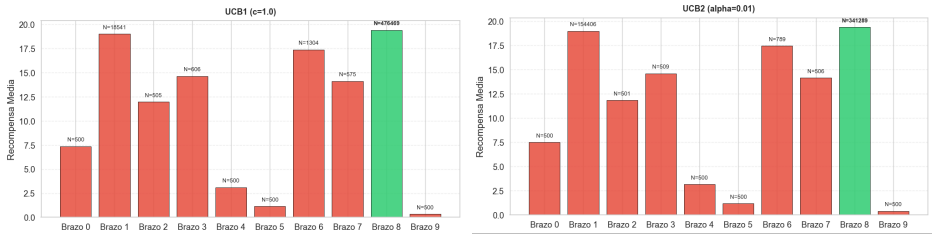

Figura 1.10: Histograma de estadísticas por brazo en el entorno Binomial (continuación).

nar con firmeza el brazo ganador en un entorno con mayor ruido y escala.

Por el contrario, la formulación de **UCB1** emerge como la estrategia más robusta. Su cota superior de confianza compensa de forma natural la escala ampliada de las recompensas empíricas, logrando la máxima rentabilidad y minimizando el arrepentimiento con una eficacia que las heurísticas probabilísticas o estáticas no pueden igualar en este nivel de complejidad.

1.4.4. Resultados: Entorno Normal

El entorno modelado mediante distribuciones Normales $\mathcal{N}(\mu, \sigma^2)$ representa el escenario más desafiante de este estudio. A diferencia de las distribuciones anteriores, las recompensas aquí son variables continuas que pueden fluctuar ampliamente alrededor de su media, introduciendo un nivel de ruido y varianza que pone a prueba la estabilidad de las estimaciones $Q(a)$ de cada algoritmo.

Recompensa Promedio

La Figura 1.11 muestra la evolución de la recompensa media, donde el óptimo teórico ronda un valor de 9.5. En este escenario continuo, **UCB1** y ϵ-Decay demuestran una rápida convergencia hacia el máximo. Sorprendentemente, el algoritmo puramente avaricioso también alcanza valores promedio muy altos de forma casi inmediata. Esto se debe a que, en la mayoría de las simulaciones, la inicialización fue suficiente para identificar un brazo "buenoz explotarlo indefinidamente.

Por el contrario, Softmax ($\tau = 0{,}1$) sufre un colapso. La combinación de varianza gaussiana y un parámetro de temperatura bajo provoca una enorme inestabilidad en las probabilidades de Gibbs, impidiendo que el algoritmo logre una explotación consistente y dejándolo estancado en una recompensa promedio muy inferior (en torno a 6.0).

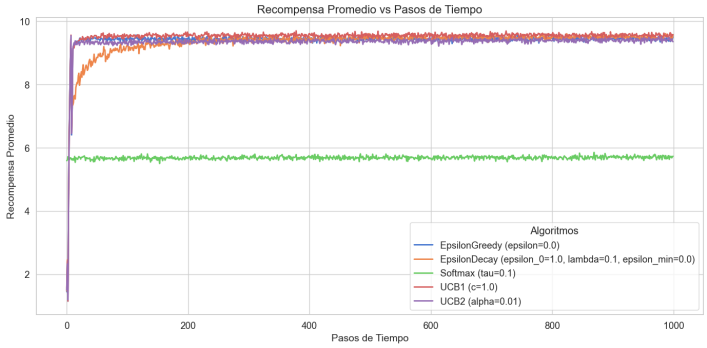

Figura 1.11: Evolución de la recompensa promedio en el entorno Normal.

Porcentaje de Selección del Brazo Óptimo

Es en la métrica de selección óptima (Figura 1.12) donde encontramos más información al respecto. La curva de ϵ-Greedy ($\epsilon = 0{,}0$) revela la trampa de la explotación pura: se estanca de forma plana alrededor del 85 %. Esto significa que en aproximadamente el 15 % de los experimentos, el agente obtuvo una recompensa inicial "desafortunada"del brazo óptimo o una inusualmente alta de un brazo subóptimo, quedando atrapado en un óptimo local para el resto de la simulación sin posibilidad de corregir su error.

Frente a esto, **UCB1** vuelve a reinar, alcanzando casi el 100 % de precisión gracias a su gestión matemática de la incertidumbre. ϵ-Decay supera con éxito la barrera del 95 % una vez su fase exploratoria concluye, demostrando que una exploración decreciente es el mejor remedio contra los óptimos locales. UCB2 ($\alpha = 0{,}01$), por su parte, exhibe una convergencia mucho más lenta, penalizado por la varianza continua que desestabiliza las estimaciones mantenidas durante sus largas épocas.

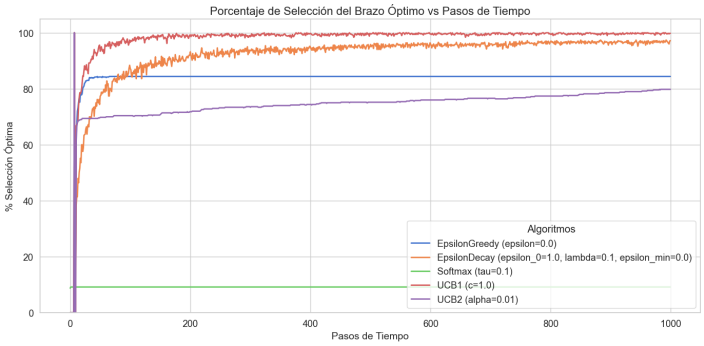

Figura 1.12: Evolución del porcentaje de selección del brazo óptimo en el entorno Normal.

Evolución del Regret Acumulado

El análisis del arrepentimiento (Figura 1.13) consolida el diagnóstico anterior. Softmax acumula un *regret* destructivo superior a 3500. Resulta de vital importancia observar la curva de ϵ-Greedy ($\epsilon = 0{,}0$): presenta una pendiente estrictamente lineal. Dado que en el 15 % de los casos nunca encuentra el brazo óptimo, sufre una pérdida constante en cada paso de tiempo, lo que teóricamente condena su *regret* a tender a infinito.

En contraposición, **UCB1** logra una curva de *regret* prácticamente plana (casi solapándose con la cota inferior teórica), lo que certifica matemáticamente su capacidad para resolver el problema minimizando las pérdidas. ϵ-Decay y UCB2 finalizan con un *regret* acumulado bajo, pero superior al de UCB1.

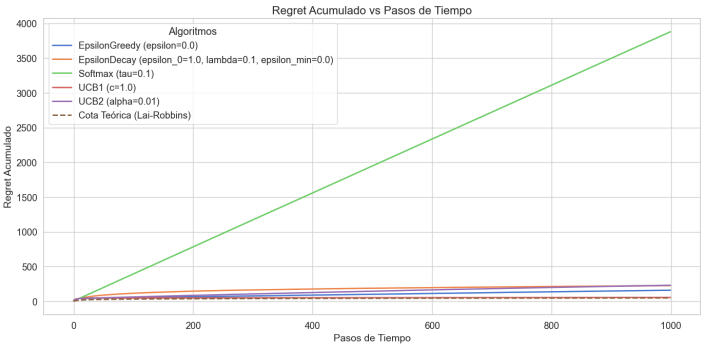

Figura 1.13: Evolución del regret acumulado en el entorno Normal.

Estadísticas por Brazo (Distribución de Selecciones)

El histograma de la Figura 1.14 permite visualizar más concretamente las decisiones de cada algoritmo. El entorno presenta su máximo global en el Brazo 7, seguido muy de cerca por el Brazo 6.

El peligro de la estrategia avariciosa ($\epsilon = 0{,}0$) queda expuesto, y es que acumula más de 77.000 selecciones en el Brazo 6. Al carecer de exploración, el agente es incapaz de discernir entre el mejor y el segundo mejor brazo si el ruido estadístico inicial le engañó. **UCB1**, en cambio, aísla el Brazo 7 con precisión (más de 486.000 selecciones), invirtiendo apenas los intentos estrictamente necesarios en el Brazo 6 para reducir su cota de incertidumbre y descartarlo. Softmax, incapaz de gestionar la varianza, distribuye sus intentos de forma casi uniforme entre los brazos de mayor valor, fracasando en su objetivo de convergencia.

Conclusiones del Entorno Normal

La experimentación sobre el entorno Gaussiano permite extraer dos conclusiones fundamentales sobre el aprendizaje por refuerzo. En primer lugar, se demuestra empíricamente el fracaso de las políticas puramente explotadoras: la ausencia de exploración garantiza la convergencia a óptimos locales en entornos con alta varianza, generando un arrepentimiento lineal e inasumible.

En segundo lugar, se ratifica la superioridad estructural de la familia de algoritmos basados en límites de confianza superior. **UCB1** se erige como la técnica más resiliente del estudio: su mecanismo de optimismo frente a la incertidumbre le permite sortear el ruido estadístico de la distribución normal, distinguir con precisión entre brazos de valores muy similares y garantizar el mínimo *regret* asintótico, todo ello sin requerir las complejas reparametrizaciones que penalizan a los métodos probabilísticos como Softmax.

1.4.5. Análisis Experimental de la Cota Teórica de Lai-Robbins

Durante la evaluación de los algoritmos en los distintos entornos estocásticos, hemos observado un fenómeno que merece una inspección teórica más profunda. En escenarios de baja varianza como el entorno de Bernoulli, algunas estrategias agresivas o en fases de exploración iniciales lograron mantener su curva de arrepentimiento (*regret*) por debajo de la cota teórica dictada por Lai y Robbins. Sin embargo, al trasladar estos mismos algoritmos a entornos de mayor escala y ruido, como las distribuciones Binomial o Normal, esta cota teórica se comportó como un límite inferior absoluto desde los primeros pasos del experimento.

Para comprender este suceso, es necesario recordar qué representa exactamente esta frontera matemática. El teorema de Lai y Robbins establece que cualquier algoritmo de asignación que sea "uniformemente bueno" sufrirá, inevitablemente, un arrepentimiento esperado L_T que crece de forma logarítmica con el tiempo. Matemáticamente, este límite asintótico viene condicionado por la divergencia de Kullback-Leibler, $KL(\mu_i, \mu^*)$, que mide la distancia de información entre la distribución de un brazo subóptimo y la del óptimo. La cota exige que el arrepentimiento cumpla la siguiente inecuación en el límite cuando el tiempo tiende a infinito:

$$L_T \geq \sum_{i:\mu_i<\mu^*} \frac{\mu^* - \mu_i}{KL(\mu_i, \mu^*)} \ln(T) \tag{1.1}$$

El propósito de este experimento aislado es ilustrar y desmitificar visualmente este aparente "incumplimiento" de la cota matemática. Para ello, hemos recreado un entorno controlado donde calculamos de forma analítica y exacta la divergencia de Kullback-Leibler para los brazos subóptimos. A través de

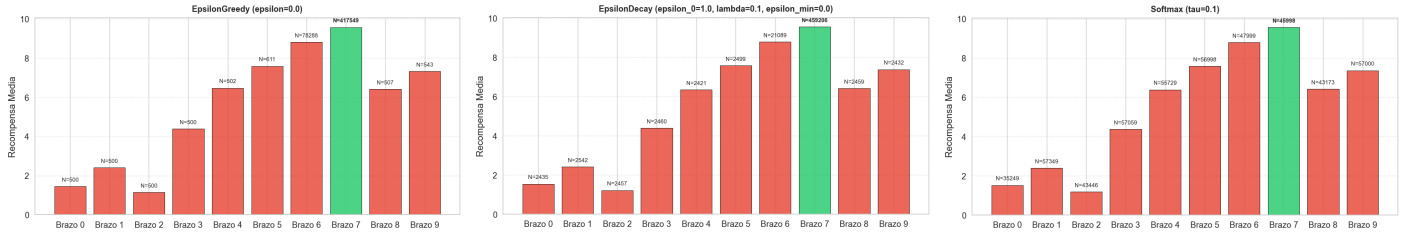

Figura 1.14: Histograma de estadísticas por brazo en el entorno Normal. Se ilustra cómo la política puramente avariciosa ($\epsilon = 0{,}0$) queda atrapada en el Brazo 6, mientras UCB1 aísla el Brazo 7.

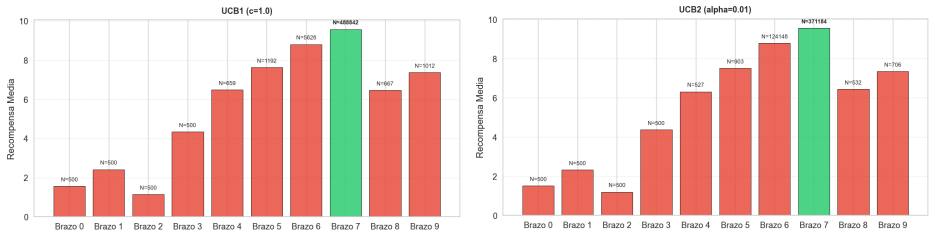

Figura 1.15: Histograma de estadísticas por brazo en el entorno Normal (continuación).

esta simulación, pretendemos demostrar que la cota de Lai-Robbins es, por definición, una garantía asintótica ($T \to \infty$), y que su cumplimiento está estrictamente subordinado a una premisa de exploración mínima.

Resultados del Experimento

Tras ejecutar la simulación controlada para los entornos de Bernoulli y Binomial, hemos extraído y tabulado el número esperado empírico de selecciones de cada brazo ($\mathbb{E}[N_i]$) frente al límite inferior teórico exigido por el teorema de Lai-Robbins, definido como $\frac{\ln(T)}{KL(\mu_i, \mu^*)}$. Los resultados obtenidos (Figura 1.16 y Figura 1.17) revelan el motivo exacto por el cual la cota de arrepentimiento parece vulnerarse a corto plazo en las gráficas generales.

	Brazo	Divergencia KL	Tiradas Esperadas
0	1	-	-
1	2	5.1017	1.35
2	3	18.5004	0.37
3	4	13.7081	0.5
4	5	20.303	0.34
5	6	8.1364	0.85
6	7	14.1995	0.49
7	8	2.7195	2.54
8	9	22.9001	0.3
9	10	10.5264	0.66

Figura 1.16: Datos sobre la cota teórica en el entorno de Bernoulli.

	Brazo	Divergencia KL	Tiradas Esperadas
0	1	-	-
1	2	5.1017	1.35
2	3	18.5004	0.37
3	4	13.7081	0.5
4	5	20.303	0.34
5	6	8.1364	0.85
6	7	14.1995	0.49
7	8	2.7195	2.54
8	9	22.9001	0.3
9	10	10.5264	0.66

Figura 1.17: Datos sobre la cota teórica en el entorno Binomial.

Análisis del Entorno Bernoulli Al observar los datos correspondientes a la distribución de Bernoulli, queda claro que el algoritmo no cumple con la premisa fundamental del teorema en este horizonte de tiempo finito. Para que la cota inferior de arrepentimiento sea aplicable, el algoritmo debería explorar los brazos subóptimos un número mínimo de veces.

Sin embargo, los datos empíricos demuestran un claro déficit exploratorio. Tomando como ejemplo el primer algoritmo y fijándonos en el Brazo 0: el agente lo selecciona 800 veces en total a lo largo de las 500 ejecuciones independientes (lo que equivale a 1.6 selecciones medias por ejecución). Por el contrario, la teoría exige un mínimo de 4.42 selecciones por ejecución para garantizar la eficiencia asintótica. Este déficit se repite de forma consistente en el resto de los brazos

subóptimos analizados.

Al no satisfacerse el requisito de exploración mínimo ($\mathbb{E}[N_i] \geq \frac{\ln(T)}{KL(\mu_i,\mu^*)}$), la consecuencia directa sobre el arrepentimiento total pierde su obligatoriedad durante la simulación. El algoritmo explora menos de lo exigido por la cota, lo que le permite acumular temporalmente un arrepentimiento inferior al límite teórico.

Análisis del Entorno Binomial Al trasladar el experimento al entorno Binomial, la situación se invierte por completo, lo que valida nuestra hipótesis inicial sobre la dependencia del requisito exploratorio.

Tomando como referencia el Brazo 6 del primer algoritmo, el límite teórico establece que se espera un mínimo de 0.49 selecciones por ejecución. Al contrastar este valor con el histograma empírico, observamos que el brazo se selecciona un total de 500 veces a lo largo del experimento (es decir, 1 vez por ejecución). Dado que 1 es estrictamente mayor que 0.49, el requisito de exploración matemática se satisface holgadamente. Al igual que en el caso anterior, esta dinámica de cumplimiento se mantiene constante para cualquier otro brazo que escojamos en este entorno.

Conclusión del Análisis Teórico

El contraste entre ambos experimentos demuestra que la validez de la cota de Lai-Robbins es dependiente del cumplimiento de su premisa de exploración mínima. La clave de que este fenómeno ocurra de forma asimétrica entre distribuciones radica en la magnitud de la divergencia de Kullback-Leibler (KL).

En entornos como el Binomial, la divergencia KL crece proporcionalmente a los parámetros de la distribución, generando un denominador muy grande que reduce el requisito exploratorio teórico a valores residuales (por ejemplo, 0.49). Dado que cualquier algoritmo necesita inicializar probando cada brazo al menos una vez, este exiguo umbral matemático es superado de forma inherente, activando la cota teórica y haciéndola actuar como un límite inferior estricto.

Por el contrario, en entornos de baja varianza informativa como Bernoulli, la reducida divergencia KL exige una mayor carga de prueba estadística (ej. 4.42 selecciones) para descartar un brazo con certeza asintótica. En horizontes de tiempo finito, los algoritmos eluden esta exigencia tomando decisiones tempranas basadas en penalizaciones drásticas: una sola recompensa nula hunde el valor estimado $Q(a)$ del brazo, provocando que el agente lo descarte casi al instante. Esta subexploración genera una ilusión gráfica de "batir"la cota de arrepentimiento, ya que el agente ahorra intentos subóptimos a corto plazo asumiendo un mayor riesgo de caer en óptimos locales.

En definitiva, la cota matemática no se vulnera; simplemente, la teoría asume un horizonte infinito ($T \to \infty$) donde las decisiones precipitadas terminan penalizando, mientras que en simulaciones finitas, las condiciones para su aplicación asintótica no siempre se alcanzan de forma empírica.

1.5. Conclusiones

El análisis realizado en este estudio confirma que no existe un algoritmo universalmente superior sin considerar la naturaleza estocástica del entorno. Los métodos basados en el optimismo frente a la incertidumbre, específicamente UCB1, han demostrado ser la estrategia más robusta y versátil. UCB1 logró minimizar el *regret* acumulado y converger al brazo óptimo de forma casi absoluta en los tres escenarios evaluados, gestionando eficientemente tanto la escala como la varianza continua del entorno Gaussiano.

Por otro lado, las estrategias heurísticas como ϵ-Decay ofrecen un buen rendimiento asintótico y evitan la convergencia a óptimos locales, pero el coste a pagar es un arrepentimiento inicial severo debido a su exploración estocástica forzada. El mayor contraste lo ofrecen los enfoques probabilísticos: Softmax demostró una extrema vulnerabilidad a la magnitud de las recompensas, colapsando en los entornos Binomial y Normal al no poder discriminar valores esperados mediante la distribución de Gibbs, y sin dar resultados muy buenos tampoco en el entorno de Bernoulli. Finalmente, el análisis de la cota de Lai-Robbins verificó que las aparentes "violaciones" de la cota a corto plazo en entornos de baja varianza se deben puramente a un déficit en el requisito de exploración mínima ($\mathbb{E}[N_i]$), reafirmando su validez como garantía asintótica.

Limitaciones del estudio y posibles mejoras

A pesar de la exhaustividad del marco experimental, este estudio presenta ciertas limitaciones que deben tenerse en cuenta. En primer lugar, la evaluación se ha restringido a entornos estrictamente estacionarios; es decir, las distribuciones subyacentes de las recompensas no fluctúan a lo largo del horizonte temporal T. En un entorno real, las preferencias o las tasas de éxito suelen variar con el tiempo.

En segundo lugar, los hiperparámetros de los algoritmos (como la constante de exploración c en UCB1, la tasa de decaimiento en ϵ-Decay, o la temperatura τ en Softmax) se han fijado de manera estática tras una validación previa. En un problema de bandido multibrazo completamente ciego, el agente no dispondría de información a priori para ajustar estos parámetros a la escala de las recompensas, lo que mermaría el desempeño de algoritmos altamente sensibles a la configuración como Softmax o UCB2.

Líneas futuras de estudio

Para continuar profundizando en la resolución del dilema de exploración-explotación, se proponen las siguientes vías de investigación futuras:

- **Entornos no estacionarios:** Modificar el banco de pruebas para que las medias de las recompensas sigan caminos aleatorios o cambien abruptamente, implementando variantes adaptativas como UCB Descontado (D-UCB) o algoritmos de ventana deslizante (*Sliding-Window UCB*).
- **Bandidos Contextuales (*Contextual Bandits*):** Expandir el modelo básico para que el agente reciba un vector de estado (contexto) antes de cada decisión, vin-

culando el problema a la personalización algorítmica y a las redes neuronales.

- **Muestreo de Thompson (*Thompson Sampling*):** Integrar y evaluar este enfoque bayesiano, que actualiza dinámicamente las distribuciones *a posteriori* de cada brazo, ofreciendo una de las alternativas modernas más potentes frente a los límites de confianza deterministas.
- **Ajuste dinámico de hiperparámetros:** Diseñar e incorporar mecanismos de meta-aprendizaje que permitan a algoritmos como Softmax adaptar su temperatura τ en tiempo de ejecución, utilizando métricas de varianza empírica recabadas durante los primeros compases del experimento.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

El aprendizaje por refuerzo es una rama del aprendizaje automático en la que un agente aprende a tomar decisiones mediante la interacción con un entorno, recibiendo recompensas o penalizaciones en función de las acciones que realiza. El objetivo del agente es aprender una política que maximice la recompensa acumulada [1, 3].

La motivación principal de este trabajo es estudiar cómo cambian los métodos de aprendizaje por refuerzo en función del entorno en el que se aplican. En concreto, se comparan algoritmos tabulares en espacios discretos y métodos aproximados en entornos donde el espacio de estados deja de poder manejarse mediante tablas explícitas.

Los objetivos son los siguientes:

- Implementar y analizar distintos algoritmos de aprendizaje por refuerzo.
- Comparar métodos *on-policy* y *off-policy*.
- Estudiar el comportamiento de métodos tabulares en un entorno discreto.
- Analizar técnicas de aproximación de funciones en un entorno continuo.
- Evaluar el rendimiento de cada agente utilizando métricas como el éxito, la longitud de los episodios y el retorno medio.

Para cumplir estos objetivos se han empleado dos entornos de la librería `Gymnasium` [2]. En primer lugar, `Taxi-v3`, que permite trabajar con métodos tabulares sobre un espacio de estados discreto y en segundo lugar con `Acrobot-v1`, cuyo espacio de observaciones es continuo y exige el uso de aproximación de funciones. En estos entornos se han entrenado y comparado los siguientes agentes: Monte Carlo On-Policy, Monte Carlo Off-Policy, SARSA, Q-Learning, SARSA semigradiente con *tile coding* y Deep Q-Learning (DQN) [1, ?, 3].

2.2. Desarrollo

2.2.1. Formulación del problema

El problema de aprendizaje por refuerzo suele modelarse como un Proceso de Decisión de Markov (MDP), definido por la tupla

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle,$$

donde [1, 3]:

- $\mathcal{S}$ es el conjunto de estados.
- $\mathcal{A}$ es el conjunto de acciones disponibles.
- $P(s'|s,a) = P(S_{t+1} = s' \mid S_t = s, A_t = a)$ representa la probabilidad de transición al estado s' cuando el agente ejecuta la acción a en el estado s.
- $R(s,a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ es la recompensa esperada asociada a dicha transición.
- $\gamma \in [0,1]$ es el factor de descuento que determina la importancia de las recompensas futuras.

La interacción entre el agente y el entorno se desarrolla de forma secuencial. En cada instante t el agente observa un estado $S_t \in \mathcal{S}$, selecciona una acción $A_t \in \mathcal{A}$ y recibe como resultado una recompensa R_{t+1} y un nuevo estado S_{t+1}. Esta interacción genera una trayectoria de la forma

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots$$

El objetivo del agente es maximizar el retorno esperado, definido como la suma de las recompensas futuras:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

En problemas de control se busca aprender la función de valor–acción asociada a una política π,

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a],$$

que representa el retorno esperado al ejecutar la acción a en el estado s y seguir posteriormente la política π. A partir de esta función puede definirse una política que seleccione, para cada estado, la acción con mayor valor estimado.

2.2.2. Métodos tabulares

Cuando el espacio de estados de un entorno es finito y no muy grande, es posible almacenar una tabla $Q(s,a)$ con los valores de cada par estado-acción. [1, 3].

Los métodos Monte Carlo actualizan sus estimaciones al final del episodio utilizando el retorno completo observado. Por el

contrario, los métodos TD (*Temporal Difference*) actualizan sus valores de forma incremental paso a paso, sin esperar al final del episodio [1, 3].

Desde el punto de vista práctico, los métodos tabulares resultan especialmente útiles cuando se quiere analizar el comportamiento del algoritmo, ya que cada actualización puede observarse directamente sobre la tabla de valores. Sin embargo, su principal limitación es que dejan de ser viables cuando el número de estados crece demasiado o cuando el espacio de estados es continuo, ya que no es posible mantener una tabla con todos los pares estado-acción.

2.2.3. Métodos aproximados

Cuando el espacio de estados es continuo o demasiado grande, las representaciones tabulares dejan de ser viables. Para solucionarlo se emplean aproximadores de funciones que permiten estimar $Q(s, a)$ como tile coding o una red neuronal.

En el primer caso, el estado continuo se transforma en un conjunto de características activas mediante *tile coding*. En el segundo, la estimación de $Q(s, a)$ se realiza con una red neuronal multicapa entrenada mediante descenso por gradiente y mejorada mediante técnicas como *experience replay* y el uso de una *target network* [1, **?**, 3].

La aproximación de funciones permite generalizar entre estados similares, algo que no ocurre en los métodos tabulares. Gracias a esta capacidad de generalización es posible trabajar con entornos continuos. Sin embargo, también introduce una mayor dificultad en el proceso de entrenamiento ya que la actualización en un estado puede afectar simultáneamente a muchos otros [3].

2.2.4. Entornos utilizados

Taxi-v3

`Taxi-v3` es un entorno de control discreto en el que un taxi debe recoger a un pasajero en una localización concreta y transportarlo a su destino dentro de una cuadrícula. El entorno representa un mapa con varias posiciones posibles para el pasajero y un destino final. El agente dispone de un conjunto finito de estados y acciones, lo que lo convierte en un escenario adecuado para comparar algoritmos tabulares.

En cada episodio el taxi comienza en una posición aleatoria del mapa y debe localizar al pasajero, recogerlo y llevarlo hasta el destino correcto. Para ello el agente dispone de acciones de movimiento (norte, sur, este y oeste) además de acciones específicas para recoger y dejar al pasajero. El entorno penaliza cada paso realizado con una pequeña recompensa negativa de -1 y penaliza más las acciones incorrectas con -10 (ej: dejar al pasajero donde no es), mientras que completar correctamente el transporte del pasajero produce una recompensa de +20.

En los notebooks se utiliza este entorno para entrenar durante 50 000 episodios con semilla fija 42 [2].

Acrobot-v1

`Acrobot-v1` representa un sistema de dos eslabones articulados. El objetivo del agente es aplicar fuerzas en las articulaciones para conseguir que el extremo final del eslabón alcance una determinada altura.

El estado del entorno está formado por variables continuas relacionadas con los ángulos de los dos eslabones y sus velocidades angulares. Debido a esta representación continua del estado, no es posible utilizar una tabla para almacenar los valores de todas las combinaciones posibles de estado y acción y hace falta un aproximador de función.

En cada paso (con valor -1) el agente puede aplicar diferentes torques sobre la articulación con el objetivo de generar el impulso necesario para elevar el brazo. El episodio termina cuando el extremo del segundo eslabón supera la altura objetivo o cuando se alcanza el número máximo de pasos permitido.

En este trabajo, los experimentos en este entorno se realizan durante 10 000 episodios con semilla 42 [2].

2.3. Algoritmos

Los pseudocódigos incluidos en esta sección se han elaborado a partir de la implementación de la práctica y del material docente de la asignatura [3, 1].

2.3.1. Monte Carlo On-Policy

El agente `AgentMonteCarloOnPolicy` 5 implementa un método Monte Carlo con política ϵ-greedy. Para ello tiene una tabla $Q(s, a)$, un contador de visitas para cada par estado-acción y una memoria con la información completa del episodio. Una vez finaliza el episodio, recorre esta información en orden inverso para calcular los retornos y actualizar los valores de la tabla. La implementación permite utilizar tanto actualización *first-visit* como *every-visit*, aunque para el análisis en este trabajo se ha utilizado la segunda opción ya que es la que esta más alineada con el resto de agentes y hace que la comparación sea más justa [1, 3].

La política de entrenamiento es ϵ-soft:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{si } a = \arg\max_{a'} Q(s, a'), \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{en otro caso.} \end{cases}$$

La actualización incremental sigue la siguiente fórmula:

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)} \left(G - Q(s, a) \right),$$

donde $N(s, a)$ es el número de visitas al par (s, a).

Este método se utiliza principalmente en tareas episódicas. Sin embargo, al necesitar episodios completos para actualizar los valores, el aprendizaje suele ser más lento que en los métodos basados en diferencias temporales.

Algorithm 5 Monte Carlo On-Policy

Require: ϵ-greedy policy, $Q(s,a)$ table initialized
for each episode **do**
  Generate a complete episode following the current policy
  Initialize $G \leftarrow 0$
  for each transition of the episode in reverse order **do**
    $G \leftarrow r + \gamma G$
    if the pair (s,a) should be updated **then**
      $N(s,a) \leftarrow N(s,a) + 1$
      $Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}(G - Q(s,a))$
    end if
  end for
end for

2.3.2. Monte Carlo Off-Policy

El agente `AgentMonteCarloOffPolicy` 6 implementa un método Monte Carlo *off-policy*. En este caso se emplea una política de comportamiento ϵ-greedy y una política objetivo greedy. Las actualizaciones se realizan mediante muestreo por importancia ponderado. Además de la tabla $Q(s,a)$, el agente tiene una tabla acumulada $C(s,a)$ donde almacena los pesos de importancia utilizados durante el aprendizaje [1, 3].

El retorno se calcula recorriendo el episodio hacia atrás y la actualización sigue la siguiente fórmula:

$$Q(s,a) \leftarrow Q(s,a) + \frac{W}{C(s,a)}\left(G - Q(s,a)\right),$$

donde W representa el peso de importancia acumulado asociado a la probabilidad de las acciones bajo la política de comportamiento.

Este método permite aprender una política objetivo distinta de la política utilizada para explorar el entorno, lo que corresponde a la característica principal de los métodos *off-policy*.

Algorithm 6 Monte Carlo Off-Policy

Require: Behavior policy b (e.g., ϵ-greedy), target policy π (greedy)
Initialize $Q(s,a) \in \mathbb{R}$ and $C(s,a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
for each episode **do**
  Generate an episode following policy b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}(G - Q(S_t, A_t))$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    if $A_t \neq \pi(S_t)$ **then**
      break
    end if
    $W \leftarrow W \cdot \frac{1}{b(A_t|S_t)}$
  end for
end for

2.3.3. SARSA

El agente `AgentSARSA` 7 implementa el algoritmo TD *on-policy* conocido como SARSA (State–Action–Reward–State–Action). Este método tiene una tabla $Q(s,a)$ y actualiza sus valores utilizando la siguiente transición observada en el entorno. En cada paso, la actualización depende de la siguiente acción seleccionada por la política ϵ-greedy [1, 3]:

$$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right].$$

Al tratarse de un método *on-policy*, SARSA aprende una política que va acorde con el comportamiento exploratorio del agente durante el entrenamiento.

Algorithm 7 SARSA

Require: $Q(s,a)$ table initialized
for each episode **do**
  Initialize S
  Choose A using an ϵ-greedy policy
  while S is not terminal **do**
    Take action A, observe R and S'
    Choose A' using an ϵ-greedy policy
    $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)]$
    $S \leftarrow S'$, $A \leftarrow A'$
  end while
end for

2.3.4. Q-Learning

El agente `AgentQLearning` 8 implementa el algoritmo TD de control *off-policy* de Q-Learning. A diferencia de SARSA, la actualización no depende de la acción realmente ejecutada en el siguiente estado, sino del valor máximo estimado entre todas las acciones posibles [1, 3]:

$$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right].$$

De esta forma, la actualización se realiza utilizando la mejor acción estimada en el siguiente estado, independientemente de la acción utilizada durante la exploración.

Al tratarse de un método *off-policy*, Q-Learning aprende directamente una política greedy óptima mientras el agente sigue explorando mediante una política ϵ-greedy. Lo que suele significar que tiene una convergencia más rápida hacia políticas eficaces en comparación con SARSA.

2.3.5. SARSA semigradiente con *tile coding*

El agente `AgentSemiGradientSARSA` 9 en lugar de almacenar una tabla estado-acción, define un aproximador lineal:

$$\hat{q}(s,a,\mathbf{w}) = \sum_{i \in \mathcal{F}(s)} w_{a,i},$$

donde $\mathcal{F}(s)$ es el conjunto de características activas del estado obtenido mediante *tile coding*.

Algorithm 8 Q-Learning

Require: $Q(s, a)$ table initialized
for each episode **do**
  Initialize state S
  while S is not terminal **do**
    Choose A using an ϵ-greedy policy
    Take action A, observe R and S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
    $S \leftarrow S'$
  end while
end for

La actualización semigradiente sigue la siguiente fórmula:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\right] \nabla \hat{q}(S, A, \mathbf{w}).$$

Como el aproximador es lineal, el gradiente vale 1 para las características activas y 0 para el resto, por lo que únicamente se actualizan los pesos correspondientes a dichas características. Esto es lo que hace que el algoritmo se llame semigradiente.

Algorithm 9 Semi-gradient SARSA with *tile coding*

Require: Approximator $\hat{q}(s, a, \mathbf{w})$, ϵ-greedy policy, learning rate α, discount factor γ
Initialize the weights $\mathbf{w}$ arbitrarily
for each episode **do**
  Initialize state S
  Choose action A using an ϵ-greedy policy based on $\hat{q}(S, \cdot, \mathbf{w})$
  while S is not terminal **do**
    Take action A, observe R and S'
    if S' is terminal **then**
      $\delta \leftarrow R - \hat{q}(S, A, \mathbf{w})$
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta\, \nabla \hat{q}(S, A, \mathbf{w})$
      break
    else
      Choose A' using an ϵ-greedy policy based on $\hat{q}(S', \cdot, \mathbf{w})$
      $\delta \leftarrow R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})$
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta\, \nabla \hat{q}(S, A, \mathbf{w})$
      $S \leftarrow S'$, $A \leftarrow A'$
    end if
  end while
end for

Aproximación de Características mediante Tile Coding Para poder trabajar con el espacio de estados continuo del entorno se ha utilizado la técnica de *tile coding*. Esta técnica permite transformar un estado continuo en un conjunto discreto de características activas.

La implementación se realiza mediante la clase `TileCoder`, que divide el espacio de estados en varias rejillas (*tilings*) ligeramente desplazadas entre sí. De esta forma, un mismo estado activa una celda distinta en cada una de las rejillas.

El proceso sigue los siguientes pasos:

1. Se define la resolución del espacio mediante el número de *tilings* y el número de divisiones por dimensión (*bins*).
2. Para un estado dado, el codificador proyecta sus coordenadas en cada una de las rejillas desplazadas.
3. En cada rejilla se activa una única celda, generando así un conjunto de índices activos.

El número total de características viene dado por

$$n_{\text{features}} = n_{\text{tilings}} \times n_{\text{bins}}^{\text{dim}},$$

donde *dim* es la dimensionalidad del estado.

Gracias a esta representación, estados similares activan características parecidas, lo que permite que el agente generalice lo aprendido a regiones cercanas del espacio de estados.

2.3.6. Deep Q-Learning

El agente `AgentDeepQLearning` 10 implementa un DQN para aproximar la función $Q(s, a)$. El agente usa una red neuronal totalmente conectada con dos capas ocultas de tamaño 12 y 8, activación ReLU y una salida con tantas componentes como acciones disponibles.

La red recibe como entrada el vector de estado del entorno y devuelve una estimación del valor de cada acción posible:

$$Q(s, \cdot) = \left(Q(s, a_1), \ldots, Q(s, a_{|\mathcal{A}|})\right).$$

De esta forma, el agente puede seleccionar la acción con mayor valor estimado siguiendo una política ϵ-greedy durante el entrenamiento.

La pérdida que se minimiza durante el entrenamiento es

$$L(\theta) = \left(Q_\theta(s, a) - y\right)^2,$$

donde el valor objetivo se calcula como

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a'),$$

siendo θ los parámetros de la red principal y θ^- los parámetros de la red objetivo.

Arquitectura de la red neuronal La función de valor se aproxima mediante una red neuronal multicapa. La red está formada por:

- Una capa de entrada que recibe el estado continuo del entorno.
- Dos capas ocultas con 12 y 8 neuronas respectivamente, utilizando activación ReLU.
- Una capa de salida con tantas neuronas como acciones disponibles, donde cada neurona representa el valor estimado $Q(s, a)$ para una acción concreta.

Esta arquitectura permite aproximar la función de valor incluso cuando el espacio de estados es continuo, algo que no sería posible con métodos tabulares.

Replay Buffer Uno de los componentes fundamentales de DQN es el *Replay Buffer*. Este mecanismo consiste en almacenar las transiciones que el agente va generando durante la interacción con el entorno. Cada transición tiene la forma

$$(s, a, r, s', done)$$

donde s es el estado actual, a la acción ejecutada, r la recompensa obtenida y s' el siguiente estado.

En lugar de entrenar la red únicamente con la transición más reciente, el algoritmo guarda muchas de estas experiencias en una memoria y posteriormente selecciona minibatches aleatorios para actualizar la red. Esta técnica nos permite reutilizar experiencias pasadas varias veces, mejorando así el aprendizaje.

Target Network Además de la red principal, DQN utiliza una segunda red llamada *target network*. Esta red es una copia de la red principal cuyos parámetros se actualizan periódicamente. Al usar esta red se estabiliza el proceso de entrenamiento, ya que el objetivo utilizado para actualizar la red principal cambia más lentamente.

Algorithm 10 Deep Q-Learning (DQN)

Require: Online network Q_θ, target network Q_{θ^-}, replay buffer $\mathcal{D}$, ϵ-greedy policy

```
Initialize the parameters θ of the online network randomly
Initialize the target network with θ⁻ ← θ
Initialize the replay buffer D
for each episode do
    Initialize state S
    while S is not terminal do
        Choose action A using an ε-greedy policy based on Q_θ(S, ·)
        Take action A, observe R, S′, and done
        Store transition (S, A, R, S′, done) in D
        if there are enough samples in D then
            Sample a minibatch of transitions from D
            for each transition (s, a, r, s′, done) in the minibatch do
                if done then
                    y ← r
                else
                    y ← r + γ máx_{a′} Q_{θ⁻}(s′, a′)
                end if
            end for
            Update θ by minimizing the loss
        end if
        Periodically update the target network: θ⁻ ← θ
        S ← S′
    end while
end for
```

En comparación con los métodos tabulares o con aproximación lineal, DQN permite representar relaciones más complejas entre estados y acciones, lo que resulta especialmente útil en entornos con espacios de estados continuos o de gran dimensión.

2.4. Evaluación/Experimentos

2.4.1. Métricas de Desempeño

Para comparar el comportamiento de los distintos agentes durante el entrenamiento, se han monitorizado tres métricas fundamentales a lo largo de los episodios, con el objetivo de evaluar tanto la capacidad de aprendizaje como la eficiencia y estabilidad de las políticas obtenidas:

1. **Proporción Acumulada de Éxitos:** Mide la fracción de episodios completados satisfactoriamente hasta el episodio t, y se define como
$$\text{ratio}(t) = \frac{\sum_{i=1}^{t} \mathbb{1}(\text{éxito}_i)}{t}.$$
Esta métrica permite observar si el agente converge hacia una política capaz de resolver el entorno de forma consistente, así como comparar qué algoritmo alcanza antes un comportamiento exitoso.
2. **Longitud del Episodio:** Representa el número de pasos que necesita el agente para finalizar un episodio. Su evolución permite analizar la eficiencia de la política aprendida, ya que, en general, un agente que desarrolla una estrategia adecuada tiende a completar la tarea en menos pasos. Además, en ciertos entornos, una menor duración del episodio puede asociarse directamente con la consecución del objetivo, como ocurre en el problema de *Acrobot*.
3. **Media Acumulada del Return:** Aproxima el rendimiento esperado de la política, relacionado con la cantidad
$$J(\pi) = \mathbb{E}[G_t].$$
Durante el entrenamiento se representa la media acumulada del retorno obtenido por el agente, lo que proporciona una visión histórica de su rendimiento. Esta métrica resulta especialmente útil para observar la tendencia global del aprendizaje y comparar la estabilidad entre distintos algoritmos.

En conjunto, estas tres métricas permiten visualizar de forma clara el comportamiento esperado en problemas de decisión secuencial modelados como MDPs: una política de mayor calidad debe maximizar el retorno esperado, aumentar progresivamente la tasa de éxito y reducir comportamientos ineficientes durante la interacción con el entorno.

2.4.2. Hiperparámetros empleados

Los experimentos se han realizado utilizando los siguientes hiperparámetros:

- **Monte Carlo On-Policy**: $\epsilon = 0{,}4$, $\gamma = 1{,}0$.
- **Monte Carlo Off-Policy**: $\epsilon = 0{,}1$, $\gamma = 1{,}0$.
- **SARSA**: tasa de exploración $\epsilon = 1{,}0$, factor de descuento $\gamma = 0{,}99$ y tasa de aprendizaje $\alpha = 0{,}1$.

- **Q-Learning**: tasa de exploración $\epsilon = 1{,}0$, factor de descuento $\gamma = 0{,}99$ y tasa de aprendizaje $\alpha = 0{,}1$.
- **SARSA semigradiente**: representación mediante *tile coding* con 8 *tilings* y 10 divisiones (*bins*) por dimensión.
- **DQN**: tasa de aprendizaje 10^{-3}, tamaño de minibatch 64 y buffer de experiencia de tamaño 10000.

En los métodos tabulares se usan los siguientes hiperparámetros: la tasa de exploración ϵ, el factor de descuento γ y, en los métodos de diferencias temporales, la tasa de aprendizaje α.

El parámetro ϵ controla la exploración del agente mediante una política ϵ-greedy. En los algoritmos **SARSA** y **Q-Learning** se ha utilizado un valor inicial de $\epsilon = 1{,}0$, lo que permite que el agente explore más el entorno al inicio del entrenamiento. Esta exploración es especialmente importante en las primeras etapas del aprendizaje, ya que el agente todavía no dispone de estimaciones fiables de la función de valor. A medida que avanza el entrenamiento, el valor de ϵ se reduce progresivamente, permitiendo que la política se vuelva cada vez más *greedy*.

En los métodos **Monte Carlo**, los valores de exploración utilizados son inferiores. En el caso de Monte Carlo On-Policy se ha utilizado $\epsilon = 0{,}4$, lo que mantiene un nivel de exploración relativamente alto para garantizar que diferentes trayectorias del entorno sean visitadas con frecuencia. En Monte Carlo Off-Policy se ha utilizado un valor menor, $\epsilon = 0{,}1$, ya que este método aprende una política objetivo distinta de la política de comportamiento y puede aproximar la política óptima incluso con una exploración más limitada.

El factor de descuento γ determina la importancia de las recompensas futuras en el cálculo del retorno. En los experimentos con `Taxi-v3` se han utilizado valores cercanos a 1 ($\gamma = 1{,}0$ en Monte Carlo y $\gamma = 0{,}99$ en SARSA y Q-Learning). Esto permite que la recompensa final obtenida al completar la tarea se propague hacia los estados anteriores, facilitando el aprendizaje de trayectorias eficientes en el entorno.

En los métodos de diferencias temporales aparece tenemos también la tasa de aprendizaje α, que determina cuánto se ajustan las estimaciones de la función de valor en cada actualización. En los experimentos se ha utilizado $\alpha = 0{,}1$, un valor medio que permite incorporar nueva información de forma progresiva sin provocar actualizaciones excesivamente grandes que puedan afectar al proceso de aprendizaje.

Por otro lado, los métodos Monte Carlo no utilizan una tasa de aprendizaje fija, ya que los valores de $Q(s, a)$ se estiman a partir del promedio de los retornos observados en los episodios. Esto implica que las actualizaciones se realizan únicamente al final de cada episodio, lo que suele producir estimaciones con mayor varianza y un aprendizaje más lento en comparación con los métodos basados en diferencias temporales.

En los métodos de **aproximación** aparecen hiperparámetros adicionales relacionados con la forma en que se representa y se entrena la función de valor.

En **SARSA semigradiente**, la función $Q(s, a)$ se aproxima mediante *tile coding*. En este trabajo se han utilizado **8 tilings** y **10 divisiones por dimensión**. El uso de múltiples *tilings* permite que cada estado esté representado por varias características activas simultáneamente, lo que mejora la capacidad de generalización entre estados cercanos.

En **Deep Q-Learning (DQN)**, la función de valor se aproxima mediante una red neuronal. En este caso se ha utilizado una tasa de aprendizaje de 10^{-3}, un valor que permite realizar actualizaciones suficientemente rápidas sin producir oscilaciones excesivas durante el entrenamiento.

Además, se ha utilizado un *replay buffer* de tamaño **10000**, lo que permite almacenar una cantidad suficiente de experiencias pasadas para entrenar la red con información más variada.

El entrenamiento se realiza utilizando **mini-batches de tamaño 64**. Este tamaño de lote permite estimar gradientes más estables que si se utilizaran transiciones individuales.

En conjunto, la elección de estos hiperparámetros permite equilibrar exploración, estabilidad del entrenamiento y velocidad de aprendizaje tanto en los métodos tabulares como en los métodos basados en aproximación de funciones. Cabe destacar que no hemos puesto como objetivo en este trabajo realizar una búsqueda de hiperparámetros, es por ello que se han escogido estos valores equilibrados.

2.4.3. Resultados en `Taxi-v3`

La Figura 2.1 muestra la proporción acumulada de éxitos en `Taxi-v3`. En esta gráfica se observa que Q-Learning y Monte Carlo Off-Policy alcanzan antes una tasa alta de episodios finalizados correctamente, lo que indica que estos algoritmos consiguen aprender una política eficaz en menos episodios. Este comportamiento va acorde con lo esperado teóricamente, ya que ambos métodos son *off-policy* y permiten aproximar directamente la política óptima mientras el agente continúa explorando el entorno.

Por su parte, SARSA también mejora progresivamente su tasa de éxito, aunque tarda más en converger. Esto se debe a que es un método *on-policy*, por lo que las actualizaciones dependen de la acción que realmente ejecuta el agente durante la exploración, haciendo que el aprendizaje sea más gradual. Monte Carlo On-Policy también logra aprender una política válida, aunque su progreso es más lento y presenta mayor variabilidad en comparación con los métodos *off-policy*, ya que las actualizaciones solo pueden realizarse al finalizar cada episodio completo.

En la Figura 2.2 se representa la longitud de los episodios. En ella se observa que, a medida que avanza el entrenamiento, los agentes aprenden a completar la tarea utilizando menos pasos. Este comportamiento refleja que las políticas aprendidas se vuelven progresivamente más eficientes. Q-Learning presenta uno de los descensos más claros en la longitud media de los episodios, lo que indica que el agente encuentra rutas más directas para recoger y entregar al pasajero. Este resultado está relacionado con el uso del operador `max` en su actualización, que permite propagar rápidamente la información de recompensa hacia los estados anteriores.

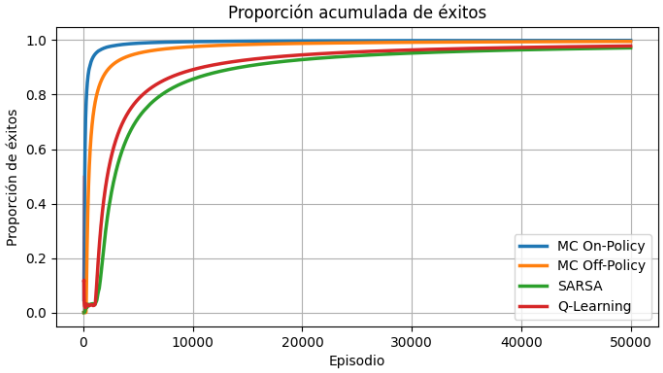

Figura 2.1: Proporción acumulada de éxitos en `Taxi-v3`.

SARSA también reduce progresivamente la duración de los episodios, aunque lo hace de forma más gradual. En todos los métodos las curvas terminan estabilizándose aproximadamente alrededor de 13–15 pasos por episodio, lo que coincide con la longitud esperada de una trayectoria óptima en `Taxi-v3`.

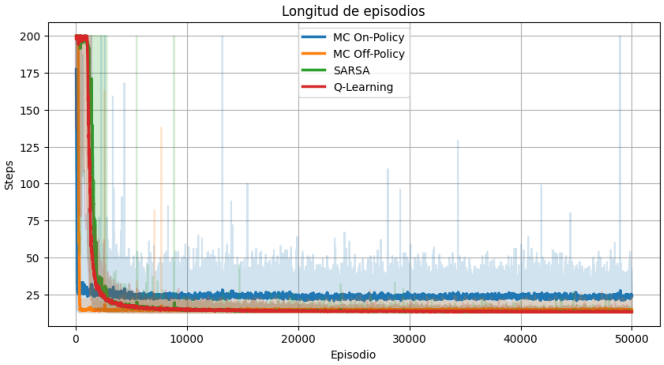

Figura 2.2: Longitud de los episodios en `Taxi-v3`.

La Figura 2.3 muestra la evolución de la media acumulada del retorno. Al inicio del entrenamiento, el retorno medio presenta valores negativos elevados, ya que el agente actúa prácticamente al azar y acumula penalizaciones por pasos innecesarios o acciones incorrectas. A medida que el agente aprende políticas más eficientes, el retorno medio mejora progresivamente.

En esta métrica se observa que Q-Learning mantiene un comportamiento estable y alcanza valores medios altos de retorno. Monte Carlo Off-Policy también mejora rápidamente su rendimiento y obtiene resultados comparables, mientras que SARSA converge de forma más gradual. Monte Carlo On-Policy presenta una evolución más lenta debido a que las actualizaciones dependen del retorno completo del episodio, lo que introduce mayor varianza en las estimaciones y ralentiza el proceso de aprendizaje en comparación con los métodos basados en diferencias temporales.

En conjunto, las tres métricas analizadas muestran un comportamiento coherente con la teoría de los algoritmos estudiados. Q-Learning y Monte Carlo Off-Policy presentan el mejor rendimiento global en este entorno, alcanzando rápidamente políticas eficaces y retornos elevados. SARSA converge correctamente aunque de forma más gradual, mientras que Monte Carlo On-Policy queda por detrás en las tres métricas analizadas debido a la necesidad de esperar al final de cada episodio

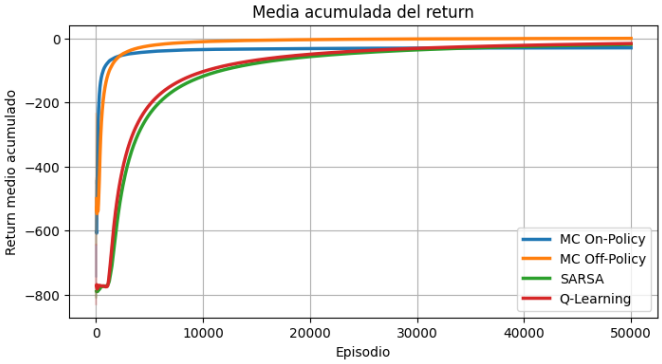

Figura 2.3: Media acumulada del return en `Taxi-v3`.

para realizar las actualizaciones.

2.4.4. Resultados en `Acrobot-v1`

La Figura 2.4 muestra la proporción acumulada de éxitos en `Acrobot-v1`. En esta gráfica se observa que Deep Q-Learning (DQN) alcanza tasas de éxito altas en menos episodios que SARSA semigradiente, lo que indica que el agente basado en redes neuronales consigue aprender una política eficaz con mayor rapidez.

Este comportamiento es coherente con las características de los algoritmos utilizados. DQN utiliza una red neuronal para aproximar la función de valor-acción $Q(s, a)$, lo que le permite representar relaciones más complejas entre estados y acciones en un entorno continuo como `Acrobot-v1`. En cambio, SARSA semigradiente utiliza tile coding y una aproximación lineal, que aunque permite generalizar entre estados similares, tiene una capacidad de representación más limitada.

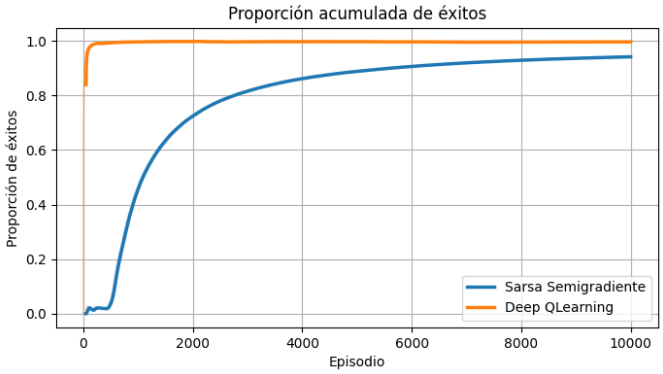

Figura 2.4: Proporción acumulada de éxitos en `Acrobot-v1`.

En la Figura 2.5 se representa la longitud de los episodios. Como cada paso tiene recompensa -1, reducir el número de pasos equivale también a mejorar el retorno acumulado. En la gráfica se observa que DQN consigue reducir antes la longitud de los episodios, lo que indica que el agente aprende más rápidamente a generar el impulso necesario para alcanzar la altura objetivo.

Este resultado está relacionado con la capacidad del modelo para capturar la dinámica del entorno. En `Acrobot-v1` el agente debe generar oscilaciones sucesivas para acumular energía y elevar el extremo del sistema por encima de la al-

tura objetivo. Aprender esta estrategia requiere capturar relaciones no lineales entre las variables del estado, algo que las redes neuronales pueden modelar con mayor facilidad que una aproximación lineal.

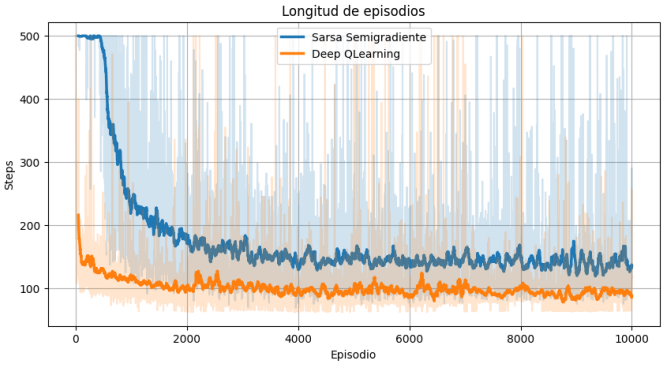

Figura 2.5: Longitud de los episodios en `Acrobot-v1`.

La Figura 2.6 confirma esta diferencia al mostrar la evolución del retorno medio durante el entrenamiento. Al inicio del aprendizaje los retornos son muy negativos, ya que el agente actúa prácticamente al azar y los episodios suelen alcanzar el número máximo de pasos.

A medida que los agentes aprenden políticas más eficientes, el número de pasos necesarios para resolver el entorno disminuye y el retorno medio mejora progresivamente. En esta métrica se observa que DQN alcanza valores menos negativos con mayor rapidez, mientras que SARSA semigradiente mejora de forma más gradual a lo largo del entrenamiento.

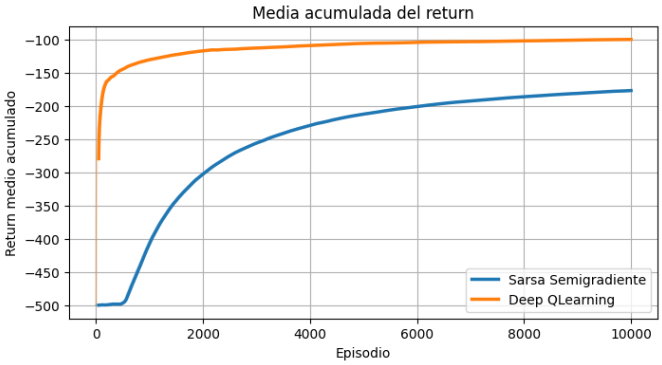

Figura 2.6: Media acumulada del return en `Acrobot-v1`.

En conjunto, las tres métricas analizadas muestran un comportamiento coherente con las propiedades de los algoritmos estudiados. Deep Q-Learning obtiene el mejor rendimiento global en este entorno, aprendiendo políticas eficaces en menos episodios y alcanzando retornos más altos.

Esta diferencia puede explicarse por la mayor capacidad de representación de las redes neuronales, que permiten aproximar funciones de valor más complejas que la aproximación lineal utilizada en SARSA semigradiente. Además, el uso de experience replay en DQN permite reutilizar experiencias pasadas y entrenar la red con mini-batches aleatorios, lo que mejora la estabilidad del aprendizaje y acelera la propagación de la información de recompensa.

2.5. Conclusiones

En este trabajo se han implementado y comparado distintos algoritmos de aprendizaje por refuerzo en dos tipos de entornos: uno discreto y tabular (`Taxi-v3`) y otro continuo (`Acrobot-v1`). Esta comparación permite observar cómo el tipo de espacio de estados influye directamente en la elección del método de aprendizaje más adecuado.

En el entorno tabular, Q-Learning y Monte Carlo Off-Policy muestran el mejor comportamiento general, alcanzando políticas eficaces con mayor rapidez que Monte Carlo On-Policy y obteniendo resultados generalmente superiores a SARSA. Por su parte, SARSA presenta un aprendizaje más gradual al tratarse de un método *on-policy*, mientras que Monte Carlo On-Policy tiende a aprender más lentamente al depender de episodios completos para realizar las actualizaciones y su naturaleza on policy lo obliga a no coger siempre la mejor acción imposible lo que dificulta su aprendizaje.

En el entorno continuo, los métodos tabulares dejan de ser aplicables y es necesario utilizar aproximación de funciones. En este contexto, SARSA semigradiente con *tile coding* permite extender los métodos TD a espacios de estados continuos mediante aproximación lineal. Sin embargo, Deep Q-Learning obtiene un mejor rendimiento gracias al uso de redes neuronales y de técnicas como *experience replay* y *target networks*, que ayudan a estabilizar el proceso de aprendizaje.

Finalmente, aunque el estudio se ha realizado sobre un número limitado de entornos y configuraciones, los resultados permiten observar claramente las diferencias entre los distintos enfoques. Como trabajo futuro, sería interesante analizar variantes más avanzadas y evaluar los métodos en entornos más complejos, además de realizar un estudio más detallado del efecto de los hiperparámetros.

En general, los resultados obtenidos muestran que no existe un único algoritmo óptimo para todos los problemas, sino que la elección del método depende en gran medida de las características del entorno y del tipo de representación del estado.

Bibliografía

[1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[2] Towers, M., Terry, J. K., Kwiatkowski, A., et al. (2023). *Gymnasium*. Farama Foundation.

[3] Material docente de la asignatura *Aprendizaje por Refuerzo*. Máster en Inteligencia Artificial, Universidad de Murcia. Diapositivas del profesor utilizadas como apoyo para los pseudocódigos.

[4] David Valera Lopez y Lucia Sanchez-Montes Gómez *ValeraSanchezMontes-RL*. Repositorio de GitHub. Disponible en: https://github.com/dac012/ValeraSanchezMontes-RL